

```

--
=====
-- 001_create_trading_platform_schema.sql
-- Team 1 - Trade Database (Postgres) | SEC3-91 / SEC3-94
--
-- Builds the full schema in one file (numbered per migrations/ convention
-- so it can also be split into 001_, 002_, ... files later and rebuilt
-- from those files alone).
--
-- Covers, per SEC3-94 acceptance criteria:
-- - FKs + CHECK constraints enforcing domain rules
-- - UNIQUE constraint on order idempotency key (DB-enforced, not
--   read-then-write, so two concurrent identical requests can't both pass)
-- - Three account states (ACTIVE / SUSPENDED / CLOSED): suspension is
--   reversible, closed accounts are never deleted (soft-close only)
-- - Instruments that stop trading are deactivated, never removed, so
--   historical orders/holdings still resolve via FK
-- - Orders can reach exactly one terminal state (SUCCESS or FAILED),
--   no half-filled state, enforced with triggers across the two
--   terminal tables
-- - version column on bank_account for optimistic concurrency, so a
--   second concurrent balance writer can detect it lost the race
--
=====

BEGIN;

-- -----
-- BANK_ACCOUNT
-- -----

CREATE TABLE bank_account (
    account_number VARCHAR(34)    PRIMARY KEY,
    name           VARCHAR(150)   NOT NULL,
    phone          VARCHAR(20),
    email          VARCHAR(150),
    balance        DECIMAL(18,2)  NOT NULL DEFAULT 0,
    bank_name      VARCHAR(150)   NOT NULL,
    ifsc_code      VARCHAR(11)    NOT NULL,

```

```

    version      INT          NOT NULL DEFAULT 0, -- optimistic concurrency on balance
updates
    CONSTRAINT chk_bank_account_balance_non_negative CHECK (balance >= 0)
);

-----
-- CLIENTS (linked_to BANK_ACCOUNT)
-----

CREATE TABLE clients (
    client_id     SERIAL        PRIMARY KEY,
    account_number VARCHAR(34)  REFERENCES bank_account(account_number),
    name          VARCHAR(150)  NOT NULL,
    email         VARCHAR(150)  NOT NULL UNIQUE,
    phone         VARCHAR(20),
    created_on    TIMESTAMP     NOT NULL DEFAULT now(),
    status        VARCHAR(10)   NOT NULL DEFAULT 'ACTIVE',
    CONSTRAINT chk_clients_status CHECK (status IN ('ACTIVE', 'SUSPENDED', 'CLOSED'))
    -- NOTE: CLOSED is a terminal state (never deleted, no hard DELETE from
    -- app code); ACTIVE <-> SUSPENDED remains reversible. Enforce the
    -- one-way transition into CLOSED at the application/service layer.
);

-----
-- AUTH (authenticates CLIENTS via email)
-----

CREATE TABLE auth (
    email         VARCHAR(150)  PRIMARY KEY REFERENCES clients(email),
    password      VARCHAR(255)  NOT NULL,
    created       TIMESTAMP     NOT NULL DEFAULT now(),
    updated       TIMESTAMP     NOT NULL DEFAULT now()
);

-----
-- INSTRUMENTS
-----

CREATE TABLE instruments (
    instrument_id SERIAL        PRIMARY KEY,
    instrument_name VARCHAR(150) NOT NULL,

```

```

is_active    BOOLEAN    NOT NULL DEFAULT TRUE
-- An instrument that stops trading is set is_active = FALSE, never
-- deleted, so historical orders/holdings can still resolve the FK.
);

-----

-- ORDERS (client places, instrument traded_in)
-----

CREATE TABLE orders (
    order_id    SERIAL    PRIMARY KEY,
    instrument_id INT      NOT NULL REFERENCES instruments(instrument_id),
    client_id   INT       NOT NULL REFERENCES clients(client_id),
    price_per_unit DECIMAL(18,4) NOT NULL,
    type        VARCHAR(4)  NOT NULL,
    quantity    INT        NOT NULL,
    exchange    VARCHAR(20) NOT NULL,
    order_timestamp TIMESTAMP NOT NULL DEFAULT now(),
    idempotency_key VARCHAR(100) NOT NULL,
    status      VARCHAR(12) NOT NULL DEFAULT 'RECEIVED',
    CONSTRAINT uq_orders_idempotency_key UNIQUE (idempotency_key),
    CONSTRAINT chk_orders_type CHECK (type IN ('BUY', 'SELL')),
    CONSTRAINT chk_orders_quantity_positive CHECK (quantity > 0),
    CONSTRAINT chk_orders_price_positive CHECK (price_per_unit > 0),
    CONSTRAINT chk_orders_status CHECK (status IN ('RECEIVED', 'IN_PROGRESS',
'SUCCESS', 'FAILED'))
);

CREATE INDEX idx_orders_client_id ON orders(client_id);
CREATE INDEX idx_orders_instrument_id ON orders(instrument_id);

-----

-- IN_PROGRESS (order generates)
-----

CREATE TABLE in_progress (
    progress_id    SERIAL    PRIMARY KEY,
    order_id       INT       NOT NULL REFERENCES orders(order_id),
    instrument_id  INT       NOT NULL REFERENCES instruments(instrument_id),
    quantity       INT       NOT NULL,

```

```

    entered_timestamp TIMESTAMP NOT NULL DEFAULT now(),
    CONSTRAINT chk_in_progress_quantity_positive CHECK (quantity > 0)
);

```

```

CREATE INDEX idx_in_progress_order_id ON in_progress(order_id);

```

```

-----
-- TRANSACTION_SUCCESS (in_progress completes_as)
-----

```

```

CREATE TABLE transaction_success (
    transaction_id SERIAL PRIMARY KEY,
    order_id INT NOT NULL UNIQUE REFERENCES orders(order_id),
    quantity INT NOT NULL,
    transaction_timestamp TIMESTAMP NOT NULL DEFAULT now(),
    value DECIMAL(18,2) NOT NULL,
    CONSTRAINT chk_txn_success_quantity_positive CHECK (quantity > 0)
);

```

```

-----
-- TRANSACTION_FAILURES (in_progress terminates_as)
-----

```

```

CREATE TABLE transaction_failures (
    transaction_id SERIAL PRIMARY KEY,
    order_id INT NOT NULL UNIQUE REFERENCES orders(order_id),
    quantity INT NOT NULL,
    transaction_timestamp TIMESTAMP NOT NULL DEFAULT now(),
    value DECIMAL(18,2) NOT NULL,
    reason_for_failure VARCHAR(255) NOT NULL
);

```

```

-- Each order_id is UNIQUE within transaction_success and within
-- transaction_failures individually (one row max per order in each).
-- The trigger below additionally prevents the SAME order_id appearing
-- in BOTH tables, so an order lands in exactly one terminal state.

```

```

CREATE OR REPLACE FUNCTION fn_enforce_single_terminal_state()
RETURNS TRIGGER AS $$
BEGIN

```

```

IF TG_TABLE_NAME = 'transaction_success' THEN
    IF EXISTS (SELECT 1 FROM transaction_failures WHERE order_id = NEW.order_id) THEN
        RAISE EXCEPTION 'order % already has a FAILED terminal state', NEW.order_id;
    END IF;
    UPDATE orders SET status = 'SUCCESS' WHERE order_id = NEW.order_id;
ELSIF TG_TABLE_NAME = 'transaction_failures' THEN
    IF EXISTS (SELECT 1 FROM transaction_success WHERE order_id = NEW.order_id) THEN
        RAISE EXCEPTION 'order % already has a SUCCESS terminal state', NEW.order_id;
    END IF;
    UPDATE orders SET status = 'FAILED' WHERE order_id = NEW.order_id;
END IF;
RETURN NEW;
END;
$$ LANGUAGE plpgsql;

```

```

CREATE TRIGGER trg_transaction_success_terminal
    BEFORE INSERT ON transaction_success
    FOR EACH ROW EXECUTE FUNCTION fn_enforce_single_terminal_state();

```

```

CREATE TRIGGER trg_transaction_failures_terminal
    BEFORE INSERT ON transaction_failures
    FOR EACH ROW EXECUTE FUNCTION fn_enforce_single_terminal_state();

```

```

-----
-- PORTFOLIO_HOLDING (client owns, instrument held_as)
-----

```

```

CREATE TABLE portfolio_holding (
    holding_id    SERIAL          PRIMARY KEY,
    client_id     INT             NOT NULL REFERENCES clients(client_id),
    instrument_id INT             NOT NULL REFERENCES instruments(instrument_id),
    quantity      INT             NOT NULL,
    avg_price     DECIMAL(18,4)   NOT NULL,
    created_at    TIMESTAMP       NOT NULL DEFAULT now(),
    updated       TIMESTAMP       NOT NULL DEFAULT now(),
    CONSTRAINT uq_portfolio_holding_client_instrument UNIQUE (client_id, instrument_id),
    CONSTRAINT chk_portfolio_holding_quantity_non_negative CHECK (quantity >= 0)
);

```

```
CREATE INDEX idx_portfolio_holding_client_id ON portfolio_holding(client_id);  
CREATE INDEX idx_portfolio_holding_instrument_id ON portfolio_holding(instrument_id);  
  
COMMIT;
```